Vortex Formation Simulation for Pulsejet Flame Stabiliza

MATLAB source listing: vortex_formation_simulation.m

568 lines | Prescribed vortex formation (growth/decay of circulation) + flame-holding Damkohler analysis

Domain: 0.3 m x 0.1 m (80 x 40 grid), 500 steps of 0.1 ms (50 ms)
Outputs: formation sequence, flame-holding maps, time-evolution plots, vortex_evolution.gif
Requires MATLAB R2018b or newer (uses sgtitle, caxis, parula, getframe).

Source reproduced verbatim from the author's file. See the README and THEORY PDFs for usage, assumptions and equation

vortex_formation_simulation.m

```
%% ========================================================================
% DETAILED FLAME STABILIZATION WITH TOROIDAL VORTEX%% ========================================================================
  ======
% VORTEX FORMATION SIMULATION FOR PULSEJET FLAME STABILIZATION
% Corrected version - No nested functions, proper MATLAB syntax
% ========================================================================

function vortex_formation_simulation()
clear; close all; clc;

fprintf('=== VORTEX FORMATION SIMULATION ===\n');

%% ==================== 1. PHYSICAL PARAMETERS ========================
L = 0.3;        % Domain length [m]
H = 0.1;        % Domain height [m]
Nx = 80;        % Grid points in x
Ny = 40;        % Grid points in y

% Create grid
x = linspace(0, L, Nx);
y = linspace(0, H, Ny);
[X, Y] = meshgrid(x, y);
dx = x(2) - x(1);
%%
%
%   for x = 1:10
%       disp(x)
%   end
%
dy = y(2) - y(1);

% Time parameters
dt = 1e-4;
T_total = 0.05;
Nt = round(T_total/dt);

% Vortex parameters
vortex_strength = 2.0;
vortex_radius = 0.04;
x_center = L/3;
y_center = H/2;
decay_rate = 0.1;

% Initialize velocity fields
U = zeros(Ny, Nx);
V = zeros(Ny, Nx);

% Add background flow (parabolic profile)
for i = 1:Nx
    for j = 1:Ny
        U(j,i) = 5.0 * (1 - ((y(j) - H/2)/(H/2))^2);
    end
end

%% ==================== 2. SIMULATION LOOP ============================
% Pre-allocate storage
time_points = zeros(1, Nt);
circulation_history = zeros(1, Nt);

% For animation storage
max_frames = 50;
U_total_history = zeros(Ny, Nx, max_frames);
V_total_history = zeros(Ny, Nx, max_frames);
vorticity_history = zeros(Ny, Nx, max_frames);
time_indices_history = zeros(1, max_frames);

frame_idx = 1;
save_interval = max(1, floor(Nt/max_frames));

fprintf('Simulating vortex formation...\n');
fprintf('Total time steps: %d\n', Nt);
tic;

for n = 1:Nt
    t = n * dt;
    time_points(n) = t;

    % Time-dependent vortex strength
    if t < 0.01
        Gamma = vortex_strength * (t/0.01);
    else
        Gamma = vortex_strength * exp(-decay_rate*(t-0.01));
    end

    % Generate vortex velocity field
    x_rel = X - x_center;
```

```
    y_rel = Y - y_center;
    r = sqrt(x_rel.^2 + y_rel.^2 + 1e-6);

    % Tangential velocity (Rankine vortex model)
    u_theta = (Gamma./(2*pi*r)) .* (1 - exp(-(r/vortex_radius).^2));

    % Convert to Cartesian (clockwise rotation)
    u_vortex = u_theta .* (-y_rel ./ r);
    v_vortex = u_theta .* (x_rel ./ r);

    % Combine with background flow
    U_total = U + u_vortex;
    V_total = v_vortex;

    % Calculate vorticity
    vorticity = zeros(Ny, Nx);
    for i = 2:Nx-1
        for j = 2:Ny-1
            du_dy = (U_total(j+1,i) - U_total(j-1,i)) / (2*dy);
            dv_dx = (V_total(j,i+1) - V_total(j,i-1)) / (2*dx);
            vorticity(j,i) = dv_dx - du_dy;
        end
    end

    % Calculate circulation
    mask = (sqrt((X - x_center).^2 + (Y - y_center).^2) < vortex_radius);
    circulation = sum(vorticity(mask)) * dx * dy;
    circulation_history(n) = circulation;

    % Store frames for animation
    if mod(n, save_interval) == 0 && frame_idx <= max_frames
        U_total_history(:,:,frame_idx) = U_total;
        V_total_history(:,:,frame_idx) = V_total;
        vorticity_history(:,:,frame_idx) = vorticity;
        time_indices_history(frame_idx) = n;
        frame_idx = frame_idx + 1;
    end

    % Display progress
    if mod(n, 1000) == 0
        fprintf('  Progress: %.1f%%\n', (n/Nt)*100);
    end
end

simulation_time = toc;
fprintf('Simulation completed in %.1f seconds\n', simulation_time);

%% ==================== 3. VISUALIZE VORTEX FORMATION ==================
figure('Position', [100, 100, 1400, 500], 'Name', 'Vortex Formation Process');
colormap(jet);

% Show key frames
num_frames_to_show = min(6, frame_idx-1);
for frame = 1:num_frames_to_show
    subplot(2, 3, frame);

    % Velocity magnitude
    U_frame = U_total_history(:,:,frame);
    V_frame = V_total_history(:,:,frame);
    vel_mag = sqrt(U_frame.^2 + V_frame.^2);

    % Visualization using surf
    surf(X, Y, vel_mag, 'EdgeColor', 'none');
    view(2); % Top view
    axis equal tight;
    shading interp;
    colorbar;
    caxis([0, 10]);

    % Get corresponding time
    n_idx = time_indices_history(frame);
    if n_idx > 0
        t_display = n_idx * dt;
        circ_display = circulation_history(n_idx);
        title(sprintf('t = %.3f s\nΓ = %.2f m²/s', t_display, circ_display));
    end

    xlabel('x [m]'); ylabel('y [m]');
    hold on;
    plot3(x_center, y_center, 15, 'wo', 'MarkerSize', 8, 'MarkerFaceColor', 'w');
    hold off;
end

sgtitle('Vortex Formation Time Sequence', 'FontSize', 14, 'FontWeight', 'bold');

%% ==================== 4. FLAME HOLDING ANALYSIS ======================
```

```
fprintf('\n=== FLAME HOLDING ANALYSIS ===\n');

% Final state
U_final = U_total_history(:,:,end);
V_final = V_total_history(:,:,end);

% Create temperature field (vortex contains hot gases)
T_hot = 2200;    % Hot combustion products [K]
T_cold = 300;    % Fresh mixture [K]
T_field = T_cold * ones(Ny, Nx);

% Vortex mask (region where vortex exists)
vortex_mask = (sqrt((X - x_center).^2 + (Y - y_center).^2) < vortex_radius);
T_field(vortex_mask) = T_hot;  % Vortex region is hot

% Calculate velocities in vortex region
vortex_vel_mag = sqrt(U_final(vortex_mask).^2 + V_final(vortex_mask).^2);
avg_vortex_velocity = mean(vortex_vel_mag);

% Residence time in vortex
if avg_vortex_velocity > 0
    residence_time = (2*pi*vortex_radius) / avg_vortex_velocity;
else
    residence_time = 1e6; % Very large if no velocity
end

% Chemical time scale (ignition delay)
A = 1e8;        % Pre-exponential factor [1/s]
Ea = 1.5e5;     % Activation energy [J/mol]
R = 8.314;      % Gas constant [J/mol·K]
T_avg_vortex = mean(T_field(vortex_mask));

if T_avg_vortex > 500
    tau_chem = 1 / (A * exp(-Ea/(R*T_avg_vortex)));
else
    tau_chem = 1e6; % Very large if too cold
end

% Damköhler number
if tau_chem > 0 && residence_time < 1e5
    Da = residence_time / tau_chem;
else
    Da = 0;
end

% Display results
fprintf('Vortex Properties:\n');
fprintf('  Diameter: %.3f m\n', 2*vortex_radius);
fprintf('  Final circulation: %.2f m²/s\n', circulation_history(end));
fprintf('  Core temperature: %.0f K\n', T_avg_vortex);
fprintf('  Average tangential velocity: %.2f m/s\n', avg_vortex_velocity);

fprintf('\nTime Scales:\n');
fprintf('  Residence time in vortex: %.2f ms\n', residence_time*1000);
fprintf('  Chemical (ignition) time: %.2f ms\n', tau_chem*1000);
fprintf('  Damköhler number (Da): %.2f\n', Da);

fprintf('\nStability Assessment:\n');
if Da > 1.5
    fprintf('  EXCELLENT flame holding (Da > 1.5)\n');
    fprintf('  Vortex provides ample residence time for ignition\n');
elseif Da > 1.0
    fprintf('  ADEQUATE flame holding (Da > 1.0)\n');
    fprintf('  Vortex can stabilize flame\n');
elseif Da > 0.5
    fprintf('  MARGINAL flame holding (0.5 < Da < 1.0)\n');
    fprintf('  Flame may be unstable under disturbances\n');
else
    fprintf('  INADEQUATE flame holding (Da < 0.5)\n');
    fprintf('  Vortex too weak to stabilize flame\n');
end

%% ==================== 5. VISUALIZE FLAME HOLDING ====================
figure('Position', [100, 100, 1400, 400], 'Name', 'Flame Holding Visualization');

% Subplot 1: Temperature field
subplot(1, 3, 1);
surf(X, Y, T_field, 'EdgeColor', 'none');
view(2);
axis equal tight;
shading interp;
colorbar;
caxis([T_cold, T_hot]);
colormap(hot);
title('Temperature Field [K]');
xlabel('x [m]'); ylabel('y [m]');
```

```
hold on;
% Draw vortex boundary
theta = linspace(0, 2*pi, 100);
plot3(x_center + vortex_radius*cos(theta), ...
      y_center + vortex_radius*sin(theta), ...
      2500*ones(size(theta)), 'w--', 'LineWidth', 2);
text(x_center, y_center, 2600, 'HOT CORE', ...
     'Color', 'white', 'FontWeight', 'bold', 'HorizontalAlignment', 'center');
hold off;

% Subplot 2: Velocity magnitude
subplot(1, 3, 2);
vel_mag_final = sqrt(U_final.^2 + V_final.^2);
surf(X, Y, vel_mag_final, 'EdgeColor', 'none');
view(2);
axis equal tight;
shading interp;
colorbar;
caxis([0, max(vel_mag_final(:))]);
colormap(jet);
title('Velocity Magnitude [m/s]');
xlabel('x [m]'); ylabel('y [m]');
hold on;
plot3(x_center, y_center, max(vel_mag_final(:))*1.1, 'wo', ...
      'MarkerSize', 10, 'MarkerFaceColor', 'w');
hold off;

% Subplot 3: Damköhler number distribution
subplot(1, 3, 3);
% Calculate Da distribution within vortex
Da_field = zeros(Ny, Nx);
for i = 1:Nx
    for j = 1:Ny
        r = sqrt((x(i)-x_center)^2 + (y(j)-y_center)^2);
        if r < vortex_radius
            u_local = sqrt(U_final(j,i)^2 + V_final(j,i)^2);
            if u_local > 0
                residence_local = (2*pi*r) / u_local;
                Da_field(j,i) = residence_local / tau_chem;
            end
        end
    end
end

surf(X, Y, Da_field, 'EdgeColor', 'none');
view(2);
axis equal tight;
shading interp;
colorbar;
caxis([0, max(Da_field(:))]);
colormap(parula);
title_str = sprintf('Damkohler Number\nAverage Da = %.2f', Da);
title(title_str);
xlabel('x [m]'); ylabel('y [m]');
hold on;
% Add Da=1 contour if present
if max(Da_field(:)) > 1
    contour3(X, Y, Da_field, [1, 1], 'r-', 'LineWidth', 2);
end
hold off;

%% ==================== 6. TIME EVOLUTION ANALYSIS ====================
figure('Position', [100, 100, 1200, 400], 'Name', 'Time Evolution Analysis');

% Subplot 1: Circulation vs time
subplot(1, 3, 1);
plot(time_points, circulation_history, 'b-', 'LineWidth', 2);
xlabel('Time [s]');
ylabel('Circulation Γ [m²/s]');
title('Vortex Strength Evolution');
grid on;
xlim([0, T_total]);
ylim([0, max(circulation_history)*1.1]);

% Subplot 2: Damköhler number evolution (estimated)
subplot(1, 3, 2);
% Calculate approximate Da over time
Da_evolution = zeros(1, Nt);
for n = 1:Nt
    if n <= length(circulation_history)
        Gamma_t = circulation_history(n);
        if Gamma_t > 0
            % Simplified model: Da ∝ sqrt(circulation)
            residence_t = (2*pi*vortex_radius) / (0.3 * sqrt(Gamma_t/vortex_radius));
            Da_evolution(n) = residence_t / tau_chem;
        end
```

```
    end
end
plot(time_points, Da_evolution, 'r-', 'LineWidth', 2);
hold on;
plot([0, T_total], [1, 1], 'k--', 'LineWidth', 1.5);
plot([0, T_total], [2, 2], 'g--', 'LineWidth', 1);
hold off;
xlabel('Time [s]');
ylabel('Damkohler Number (Da)');
title('Flame Stability Evolution');
legend('Da(t)', 'Stability threshold', 'Safe margin', 'Location', 'best');
grid on;
xlim([0, T_total]);
if max(Da_evolution) > 0
    ylim([0, max(Da_evolution)*1.1]);
end

% Subplot 3: Energy analysis
subplot(1, 3, 3);
rho = 1.2;  % Air density [kg/m³]
kinetic_energy = zeros(1, Nt);
vortex_energy = zeros(1, Nt);

for n = 1:Nt
    % For energy calculation, use every 10th frame to save time
    if mod(n, 10) == 0 || n == Nt
        % Find closest stored frame
        [~, frame_idx_est] = min(abs(time_indices_history - n));
        if frame_idx_est <= size(U_total_history, 3)
            U_t = U_total_history(:,:,frame_idx_est);
            V_t = V_total_history(:,:,frame_idx_est);

            % Total kinetic energy
            KE_density = 0.5 * rho * (U_t.^2 + V_t.^2);
            kinetic_energy(n) = sum(KE_density(:)) * dx * dy;

            % Vortex energy (within vortex region)
            if any(vortex_mask(:))
                vortex_KE = 0.5 * rho * (U_t(vortex_mask).^2 + V_t(vortex_mask).^2);
                vortex_energy(n) = sum(vortex_KE(:)) * dx * dy;
            end
        end
    end
end

% Interpolate to fill gaps
non_zero_idx = kinetic_energy > 0;
if sum(non_zero_idx) > 2
    kinetic_energy = interp1(find(non_zero_idx), kinetic_energy(non_zero_idx), 1:Nt, 'linear', 'extrap');
    vortex_energy = interp1(find(vortex_energy>0), vortex_energy(vortex_energy>0), 1:Nt, 'linear', 'extrap');
end

plot(time_points, kinetic_energy, 'b-', 'LineWidth', 2);
hold on;
plot(time_points, vortex_energy, 'r-', 'LineWidth', 2);
hold off;
xlabel('Time [s]');
ylabel('Energy [J]');
title('Energy Distribution');
legend('Total kinetic energy', 'Vortex energy', 'Location', 'best');
grid on;
xlim([0, T_total]);

%% ==================== 7. CREATE ANIMATION ============================
fprintf('\nCreating animation...\n');

fig_anim = figure('Position', [100, 100, 1000, 400], 'Visible', 'off');

num_anim_frames = min(20, frame_idx-1);
anim_indices = round(linspace(1, frame_idx-1, num_anim_frames));

for anim_count = 1:num_anim_frames
    idx = anim_indices(anim_count);

    % Velocity magnitude
    subplot(1, 2, 1);
    U_anim = U_total_history(:,:,idx);
    V_anim = V_total_history(:,:,idx);
    vel_mag_anim = sqrt(U_anim.^2 + V_anim.^2);

    surf(X, Y, vel_mag_anim, 'EdgeColor', 'none');
    view(2);
    axis equal tight;
    shading interp;
    colorbar;
    caxis([0, 10]);
```

```
    colormap(jet);

    t_anim = time_indices_history(idx) * dt;
    title(sprintf('Velocity at t = %.3f s', t_anim));
    xlabel('x [m]'); ylabel('y [m]');

    % Vorticity
    subplot(1, 2, 2);
    vort_anim = vorticity_history(:,:,idx);

    surf(X, Y, vort_anim, 'EdgeColor', 'none');
    view(2);
    axis equal tight;
    shading interp;
    colorbar;
    colormap(jet);

    title(sprintf('Vorticity at t = %.3f s', t_anim));
    xlabel('x [m]'); ylabel('y [m]');

    drawnow;

    % Save frame
    frame_img = getframe(fig_anim);
    [imind, cm] = rgb2ind(frame_img.cdata, 256);

    if anim_count == 1
        imwrite(imind, cm, 'vortex_evolution.gif', 'gif', ...
                'Loopcount', inf, 'DelayTime', 0.1);
    else
        imwrite(imind, cm, 'vortex_evolution.gif', 'gif', ...
                'WriteMode', 'append', 'DelayTime', 0.1);
    end
end

close(fig_anim);
fprintf('Animation saved as vortex_evolution.gif\n');

%% ==================== 8. DESIGN RECOMMENDATIONS ====================
fprintf('\n=============================================\n');
fprintf('DESIGN RECOMMENDATIONS\n');
fprintf('=============================================\n\n');

% Optimal vortex size recommendation
optimal_vortex_ratio = 0.6;  % Vortex diameter / Chamber diameter
chamber_diameter = H;
optimal_vortex_diameter = optimal_vortex_ratio * chamber_diameter;
optimal_vortex_radius = optimal_vortex_diameter / 2;

fprintf('1. VORTEX SIZE:\n');
fprintf('   Current vortex diameter: %.3f m\n', 2*vortex_radius);
fprintf('   Optimal vortex diameter: %.3f m (%.0f%% of chamber)\n', ...
        optimal_vortex_diameter, optimal_vortex_ratio*100);

% Check if size is optimal
size_diff = abs(2*vortex_radius - optimal_vortex_diameter);
if size_diff < 0.005
    fprintf('   Recommendation: Size is optimal\n');
else
    fprintf('   Recommendation: Adjust vortex size toward optimal\n');
end

% Circulation requirement
if Da > 1
    required_circulation_multiplier = 1.0;
else
    required_circulation_multiplier = 1.5/Da;
end
required_circulation = vortex_strength * required_circulation_multiplier;

fprintf('\n2. VORTEX STRENGTH:\n');
fprintf('   Current circulation: %.2f m²/s\n', circulation_history(end));
fprintf('   Required for Da>1: %.2f m²/s\n', required_circulation);

% Check if strength is adequate
if circulation_history(end) >= required_circulation
    fprintf('   Status: Adequate\n');
else
    fprintf('   Status: Increase vortex strength by %.0f%%\n', ...
            (required_circulation/circulation_history(end) - 1)*100);
end

% Position recommendation
optimal_position_ratio = 0.35;  % Position from head / total length
optimal_position = optimal_position_ratio * L;

```

```
fprintf('\n3. VORTEX POSITION:\n');
fprintf('   Current position: %.3f m from head (%.0f%%)\n', ...
        x_center, (x_center/L)*100);
fprintf('   Optimal position: %.3f m from head (%.0f%%)\n', ...
        optimal_position, optimal_position_ratio*100);

% Flame holding effectiveness
fprintf('\n4. FLAME HOLDING EFFECTIVENESS:\n');
fprintf('   Damköhler number: Da = %.2f\n', Da);

if Da > 1.5
    fprintf('   Rating: EXCELLENT\n');
    fprintf('   Vortex provides robust flame stabilization\n');
elseif Da > 1.0
    fprintf('   Rating: ADEQUATE\n');
    fprintf('   Vortex can stabilize flame under normal conditions\n');
elseif Da > 0.7
    fprintf('   Rating: MARGINAL\n');
    fprintf('   Flame may be unstable during transients\n');
    improvement_needed = (1/Da - 1)*100;
    fprintf('   Consider increasing vortex strength by %.0f%%\n', improvement_needed);
else
    fprintf('   Rating: INADEQUATE\n');
    fprintf('   Vortex cannot stabilize flame\n');
    improvement_needed = (1/Da - 1)*100;
    fprintf('   Required improvement: %.0f%% increase in circulation\n', improvement_needed);
end

% Energy efficiency
vortex_energy_fraction = 0;
if kinetic_energy(end) > 0
    vortex_energy_fraction = vortex_energy(end) / kinetic_energy(end);
end

fprintf('\n5. ENERGY EFFICIENCY:\n');
fprintf('   Vortex energy / Total kinetic energy: %.1f%%\n', ...
        vortex_energy_fraction * 100);
if vortex_energy_fraction < 0.3
    fprintf('   Rating: Efficient use of energy\n');
else
    fprintf('   Rating: High energy consumption by vortex\n');
    fprintf('   Consider optimizing vortex formation\n');
end

fprintf('\n==========================================\n');
if Da > 1
    fprintf('SUMMARY: VORTEX DESIGN IS EFFECTIVE FOR FLAME STABILIZATION\n');
else
    fprintf('SUMMARY: VORTEX DESIGN NEEDS IMPROVEMENT\n');
end
fprintf('==========================================\n');

end
```